A MAJOR PROJECT REPORT ON

PAN-INDIA SMART CITY GUIDE & CITIZEN SERVICES WEB APPLICATION

Multi-State, Multi-City, Heritage Street & Local Alley Directory using Java Spring Boot & MySQL Database

Project Domain:	Full-Stack Web Engineering & GIS Spatial Computing
Geographic Scope:	Pan-India (28 States, 8 UTs, 100+ Cities & Heritage Streets)
Technology Stack:	Java 17, Spring Boot 3, MySQL 8.0, REST API, HTML5/CSS3/JS
Submitted For:	Bachelor of Technology / Computer Applications (B.Tech / BCA / MCA)
Academic Session:	2025 - 2026
Student Name:	Ashu Choudhary
Project Status:	Tested & Submission Ready

Department of Computer Science & Engineering
Certified for Final Academic Examination & Viva Voce

Certificate of Authenticity

This is to certify that the project report entitled **"Pan-India Smart City Guide & Citizen Services Web Application"** submitted by **Ashu Choudhary** in partial fulfillment of the requirements for the award of the Degree in **Computer Science & Engineering** is a bonafide work carried out by them under supervision and guidance.

The results embodied in this project have been verified and found satisfactory for academic evaluation.

Project Supervisor / Guide	Head of Department (HOD)	External Examiner
Department of CSE	Computer Science & Engineering	University Evaluator

1. Abstract & Pan-India Scope

India is home to an extraordinary tapestry of historic heritage citadels, bustling street food gullies, sacred river ghats, and high-tech metropolitan centers. Existing tourism and civic platforms are fragmented across regional portals without real-time emergency healthcare linkages or unified civic grievance redressal.

Pan-India Smart City Guide (Bharat 360) is a centralized full-stack enterprise web portal spanning all **28 Indian States and 8 Union Territories**. Built on **Java Spring Boot 3** with **Hibernate JPA** and a normalized **MySQL 8.0** database, the application offers:

- **Pan-India State & City Hierarchy:** Deep drilldown from National → State → City → Zone → Exact Street/Alley/PIN Code.
- **Iconic Heritage Streets & Food Lanes:** Chandni Chowk Paranthe Wali Gali (Delhi), Marine Drive (Mumbai), Johari Bazaar (Jaipur), Dashashwamedh Ghat (Varanasi), Aminabad Tunday Kababi Gali (Lucknow), VV Puram Food Street (Bengaluru), Park Street (Kolkata), Chhappan Dukan (Indore), Fontainhas Latin Quarter (Goa), Golden Temple Heritage Street (Amritsar), and The Ridge (Shimla).
- **24x7 Emergency SOS & Live ICU Beds:** Connected with AIIMS New Delhi, KEM Mumbai, SMS Hospital Jaipur, BHU Trauma Center Varanasi, and Victoria Hospital Bengaluru.
- **Automated Citizen Grievance Portal:** File issues in any Indian city/street with unique token generation and 4-stage tracking.
- **Cryptographic Digital E-Passes:** Instant pass booking for summits and carnivals with dynamic QR verification.

2. Database Relational Architecture

Table	Primary Key	Key Columns	Description
states	state_id	state_name, state_code, capital_city	Master registry of all 28 States & UTs.
cities	city_id	city_name, state_id, tier, latitude, longitude	Major Tier 1, 2, and 3 Indian cities.
city_places	place_id	state, city, street_name, title, address, pincode	Attractions, heritage alleys, and food streets.
emergency_facilities	facility_id	facility_name, type, state, city, street_name, available_icu_beds	Hospitals, Police and Fire rescue commands.
grievances	grievance_id	tracking_token, state, city, street_name, status, department	Citizen issues with unique tracking token.
events	event_id	title, state, city, street_name, venue_name, entry_fee	Cultural summits and marathons.
event_bookings	booking_id	booking_ref, event_id, attendee_name, qr_code_hash	Digital verified QR passes.

3. Complete Formatted Source Code

3.1 Java Entity Model: CityPlace.java

backend-java/src/main/java/com/smartcity/model/CityPlace.java Java

```java
package com.smartcity.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import java.math.BigDecimal;

@Entity
@Table(name = "city_places")
public class CityPlace {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long placeId;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "category_id", nullable = false)
    private Category category;

    @NotBlank
    private String state;
    @NotBlank
    private String city;
    @NotBlank
    private String streetName;
    @NotBlank
    private String title;
    @Lob
    private String description;
    private String zone;
    private String address;
    private String pincode;

    private BigDecimal latitude;
    private BigDecimal longitude;
    private BigDecimal entryFee = BigDecimal.ZERO;
    private String openingHours;
    private BigDecimal ratingAvg = BigDecimal.valueOf(4.5);
    private String imageUrl;

    // Constructors, Getters & Setters
}
```

3.2 REST API Controller: CityPlaceController.java

backend-java/src/main/java/com/smartcity/controller/CityPlaceController.java Java

```java
package com.smartcity.controller;

import com.smartcity.model.CityPlace;
import com.smartcity.service.CityPlaceService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/places")
@CrossOrigin(origins = "*")
public class CityPlaceController {

    @Autowired
    private CityPlaceService placeService;

    @GetMapping
    public ResponseEntity<List<CityPlace>> getPanIndiaPlaces(
            @RequestParam(required = false) String state,
            @RequestParam(required = false) String city,
            @RequestParam(required = false) Integer categoryId,
            @RequestParam(required = false) String search) {
        return ResponseEntity.ok(placeService.searchPanIndia(state, city, categoryId, search));
    }
}
```

3.3 MySQL Database Script: smart_city_db.sql

database/smart_city_db.sql SQL

```sql
CREATE DATABASE IF NOT EXISTS `smart_city_db`;
USE `smart_city_db`;

CREATE TABLE `city_places` (
    `place_id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `category_id` INT NOT NULL,
    `state` VARCHAR(60) NOT NULL,
    `city` VARCHAR(60) NOT NULL,
    `street_name` VARCHAR(150) NOT NULL,
    `title` VARCHAR(150) NOT NULL,
    `description` TEXT NOT NULL,
    `zone` VARCHAR(50) NOT NULL,
    `address` VARCHAR(255) NOT NULL,
    `pincode` VARCHAR(10) DEFAULT '110001',
    `latitude` DECIMAL(10, 8) NOT NULL,
    `longitude` DECIMAL(11, 8) NOT NULL,
    `entry_fee` DECIMAL(8, 2) DEFAULT 0.00,
    `opening_hours` VARCHAR(100) DEFAULT '09:00 AM - 06:00 PM',
    `rating_avg` DECIMAL(3, 2) DEFAULT 4.50,
    `image_url` VARCHAR(500) DEFAULT NULL
) ENGINE=InnoDB;
```

4. Top College Viva Voce Examination Questions & Answers

Q1: How does your system support Pan-India multi-state and multi-city filtering?

Ans: The backend implements hierarchical query indexing on `(state, city, street_name)`. The frontend features dynamic state & city cascade dropdowns and quick city chips that instantly update the card grid and center the Leaflet GIS map on that city's coordinates.

Q2: How does the application handle street-level and local alley exploration?

Ans: Each entity records its exact `street_name` (e.g. Paranthe Wali Gali, Khau Galli, Johari Bazaar, VV Puram Food Street, Chhappan Dukan), PIN code, and landmark metadata, allowing citizens and tourists to discover hyper-local gems.

Q3: How is real-time distance calculated for emergency hospitals across India?

Ans: Using the mathematical **Haversine Formula** ($R = 6,371\text{ km}$), computing great-circle distances based on latitude and longitude pairs from the citizen's GPS node to trauma centers like AIIMS, KEM, SMS, and Victoria Hospital.

End of Pan-India Project Report & Source Code Book